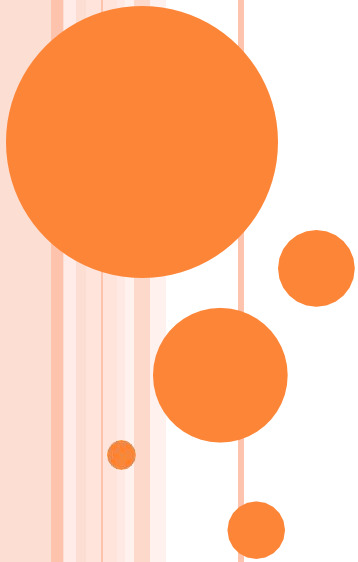


**Structures de  
Données**

**CHAPITRE V:  
ARBRES BINAIRES**



# PLAN DU CHAPITRE V

- ❑ Introduction
- ❑ Définitions et Terminologies
- ❑ Les Arbres Binaires
- ❑ Les Arbres Binaires de Recherche (ABR)
- ❑ Les Arbres AVL

# INTRODUCTION

## ❖ Dans les tableaux, nous avons :

- Un accès direct par indice (rapide)
- L'insertion et la suppression nécessitent des décalages

## ❖ Dans les Listes Linéaires Chaînées, nous avons :

- Un accès séquentiel lent
- L'insertion et la suppression se font uniquement par modification de chaînage

## ❖ Les arbres représentent un compromis entre les deux :

- Un accès relativement rapide à un élément à partir de sa clé
- L'insertion et la suppression non coûteuses

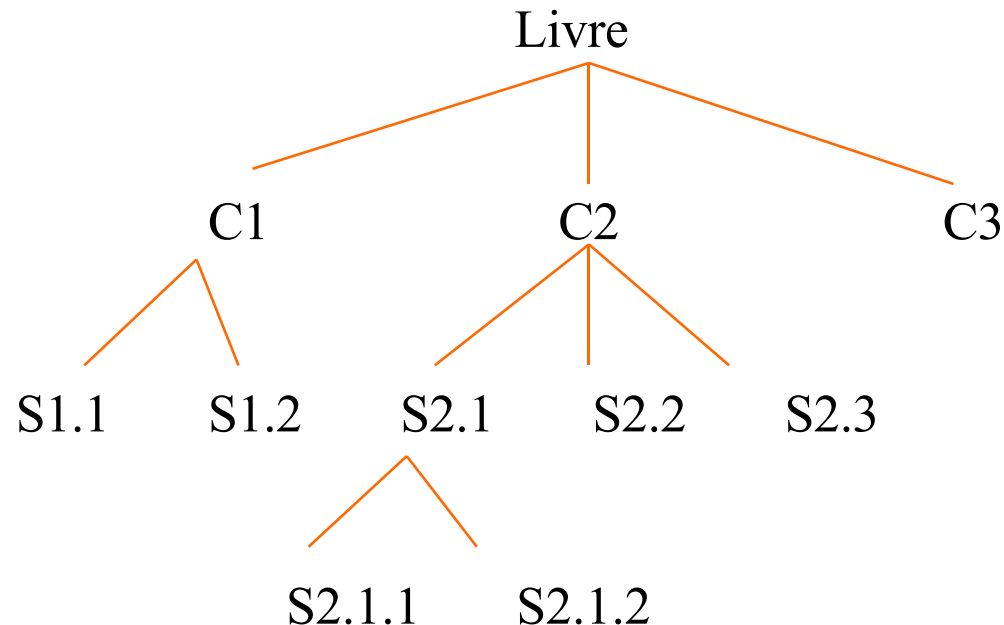
# INTRODUCTION

- ❖ De plus, les arbres sont des structures de données fondamentales en informatique, très utilisés dans tous les domaines, parce qu'ils sont bien adaptés à la représentation naturelle d'informations homogènes organisées, et d'une grande commodité et rapidité de manipulation.

# INTRODUCTION

❖ Leur usage est multiple, car il capte l'idée de hiérarchie; à titre d'exemples, nous pouvons citer:

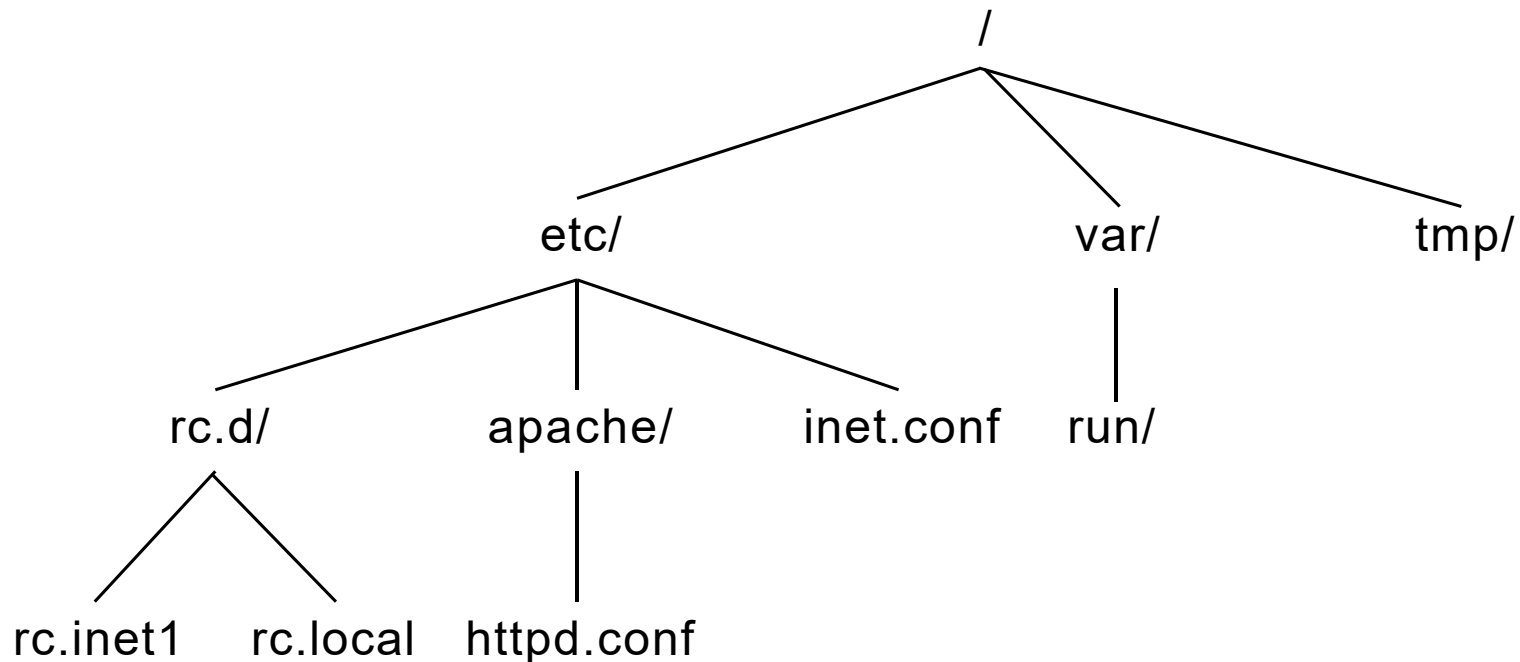
❖ Découpage d'un livre en parties, chapitres, sections, paragraphes...,



# INTRODUCTION

❖ Leur usage est multiple, car il capte l'idée de hiérarchie; à titre d'exemples, nous pouvons citer:

❖ Hiérarchies de fichiers,



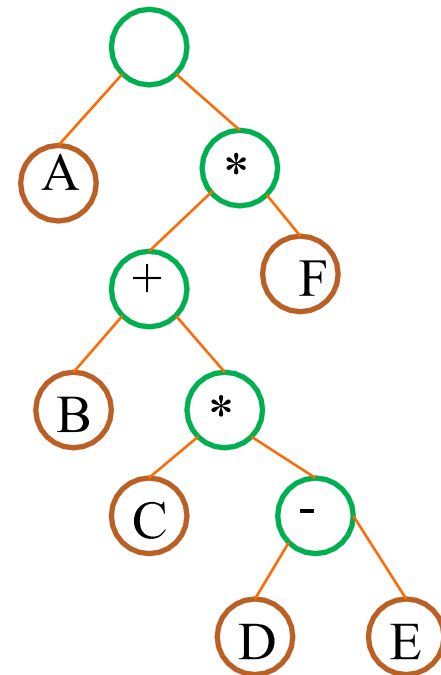
# INTRODUCTION

- ❖ Leur usage est multiple, car il capte l'idée de hiérarchie; à titre d'exemples, nous pouvons citer:

- ❖ Expressions Arithmétiques

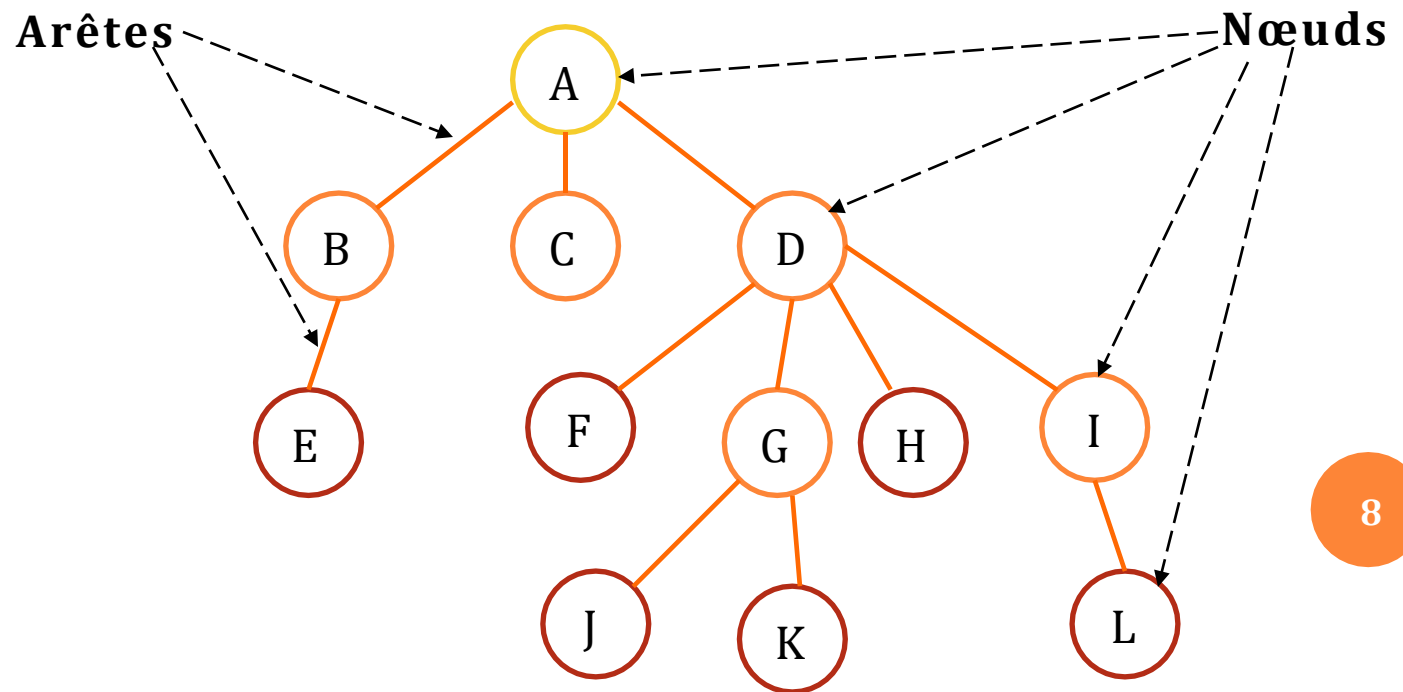
■

$A \quad B \quad C \quad D \quad E \quad F$



# DÉFINITIONS & TERMINOLOGIES

❖ Un arbre est une structure de données (souvent dynamique) représentant un ensemble de valeurs organisées hiérarchiquement (non linéaire). Chaque valeur est stockée dans un **nœud**. Les nœuds sont connectés entre eux par des **arêtes** qui représentent des relations parent/fils.



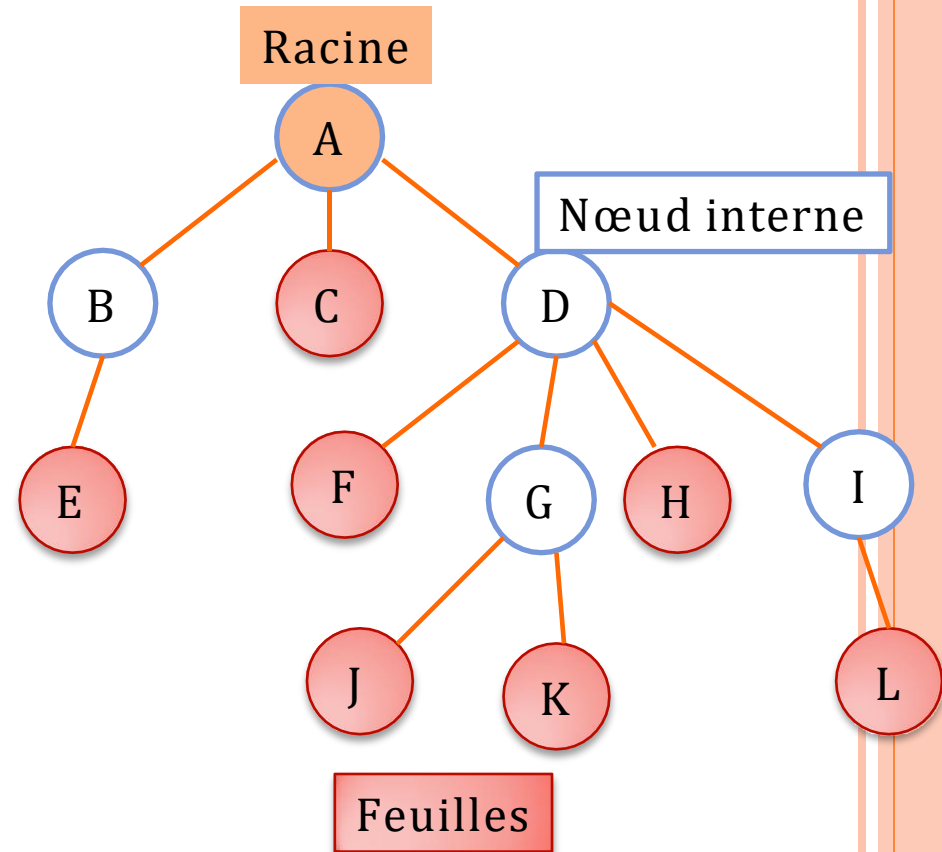


# DÉFINITIONS & TERMINOLOGIES

❖ **Racine**: est le nœud qui n'a pas de **prédécesseur** (parent) et possède zéro ou plusieurs **fils**. La racine constitue la caractéristique d'un arbre.

❖ **Feuille** : est un nœud qui n'a pas de successeur (fils). Une feuille est aussi appelée un nœud externe.

❖ **Nœud interne** : est tout nœud qui admet au moins un successeur (fils).

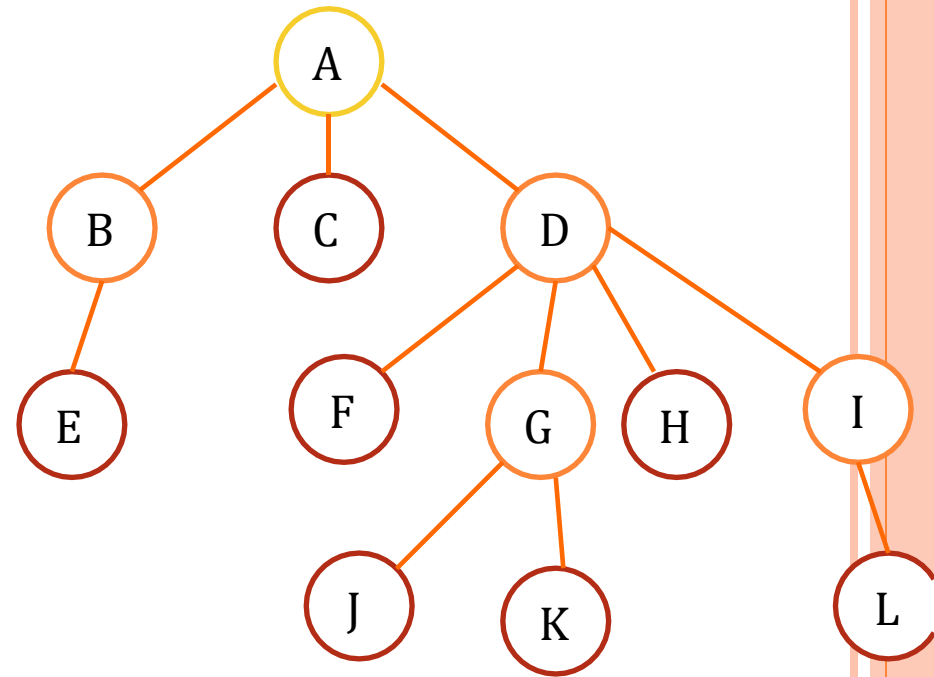


# DÉFINITIONS & TERMINOLOGIES

❖ **Fils d'un nœud** : sont ses successeurs. Dans l'exemple, F, G, H, et I sont les fils du nœud D.

❖ **Frères** : sont les successeurs ou les fils issus d'un même nœud (parent direct). Dans l'exemple, B, C et D sont des frères.

❖ **Père** : est un nœud qui admet au moins un successeur (fils). Dans l'exemple, D est le père des nœuds F, G, H et I.



# DÉFINITIONS & TERMINOLOGIES

❖ **Sous arbre** : est une portion de l'arbre. Dans l'exemple, le nœud G avec ces deux fils J et K constituent un sous arbre.

❖ **Une branche** est une suite de nœuds connectés de père en fils (de la racine à une feuille).

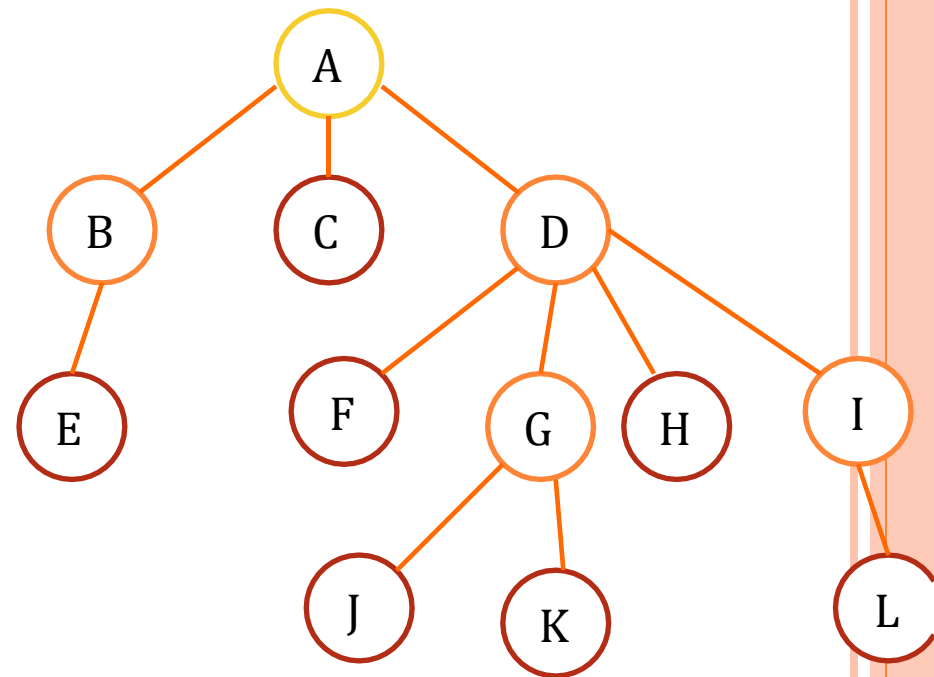
➤ A-B-E

➤ A-C

➤ A-D-F

➤ A-D-G-J

➤ .....



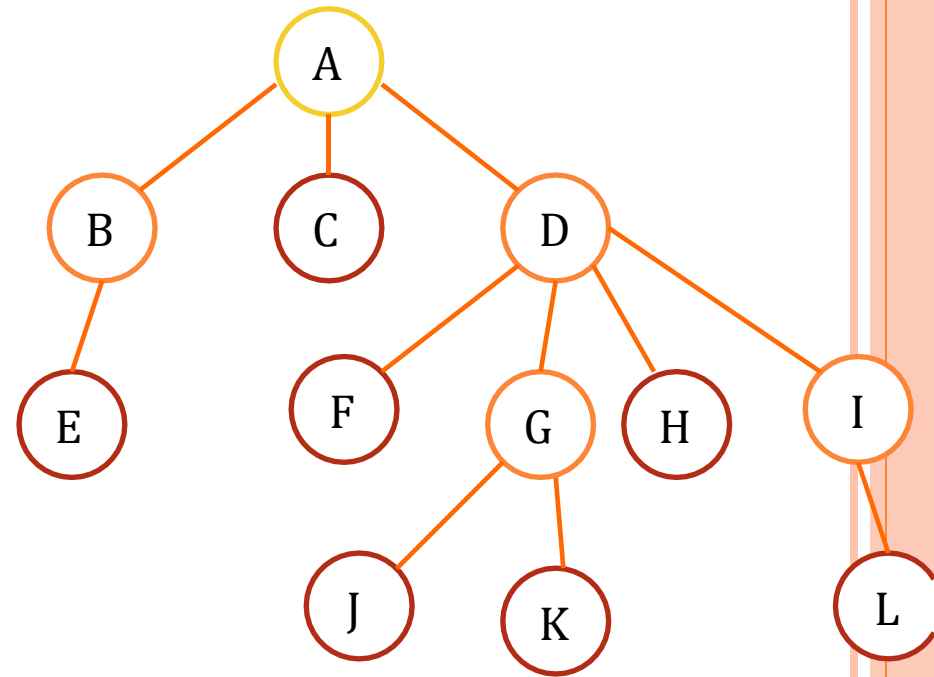
# DÉFINITIONS & TERMINOLOGIES

## ❖ Descendants d'un nœud :

sont tous les nœuds du sous arbre de racine nœud. Dans l'exemple, les descendants de D sont F, G, H, I, J, K et L.

## ❖ Ascendants d'un nœud :

sont tous les nœuds se trouvant sur la branche de la racine vers ce nœud. Dans l'exemple, les ascendants de J sont G, D et A. Les ascendants de E sont B et A.



# DÉFINITIONS & TERMINOLOGIES

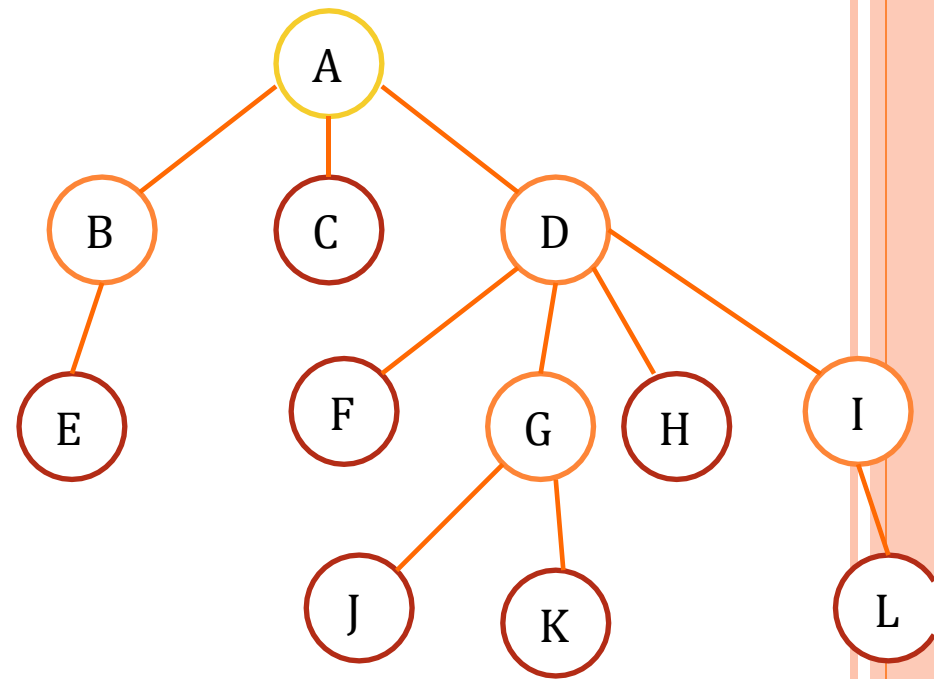
❖ **Taille d'un arbre:** est le nombre de nœuds qu'il possède.

- Taille de l'arbre ci contre = 12
- Un arbre vide est de taille égale à 0.

❖ **Degré d'un nœud :** est le nombre de ses fils. Dans l'exemple, le degré de B est 1, le degré de D est 4.

❖ **Degré d'un arbre :** est le degré maximum de ses nœuds.

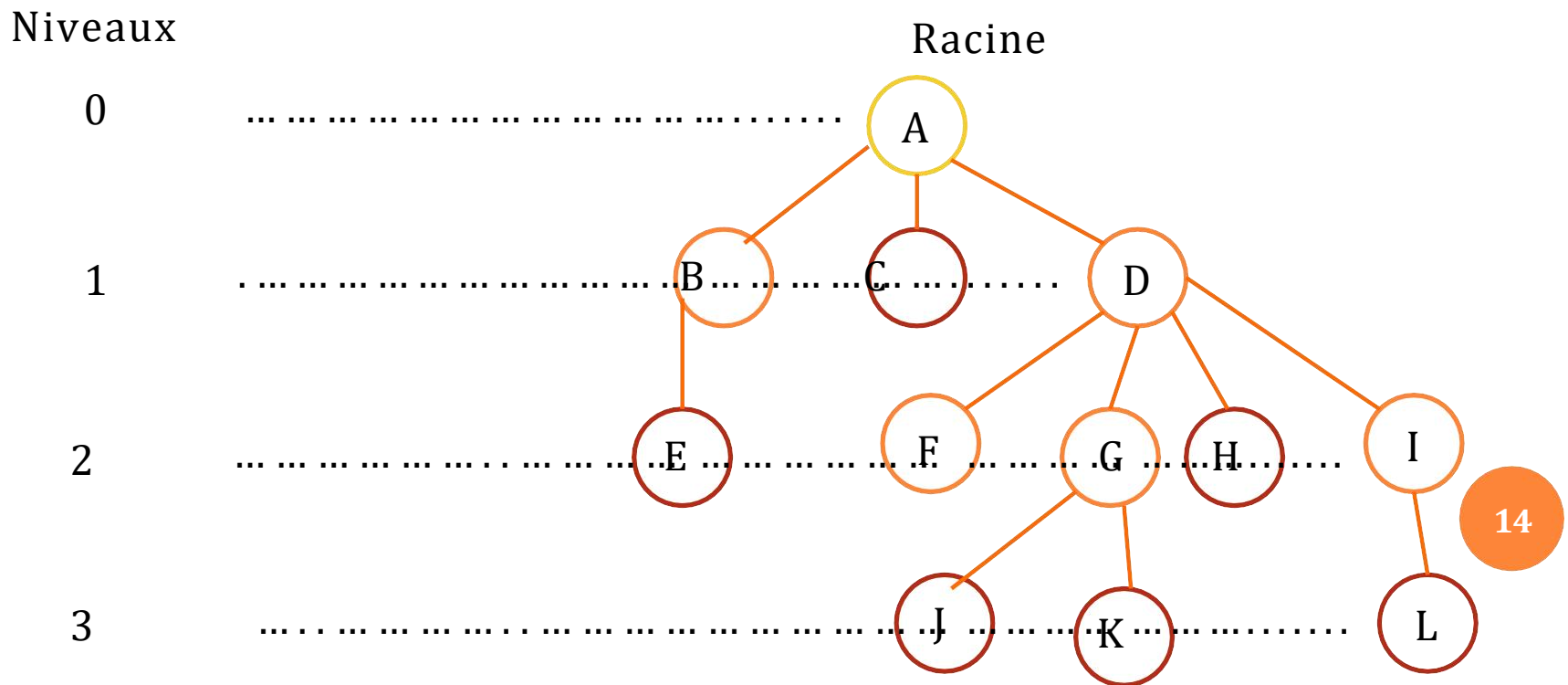
- Degré de l'arbre ci contre = 4



# DÉFINITIONS & TERMINOLOGIES

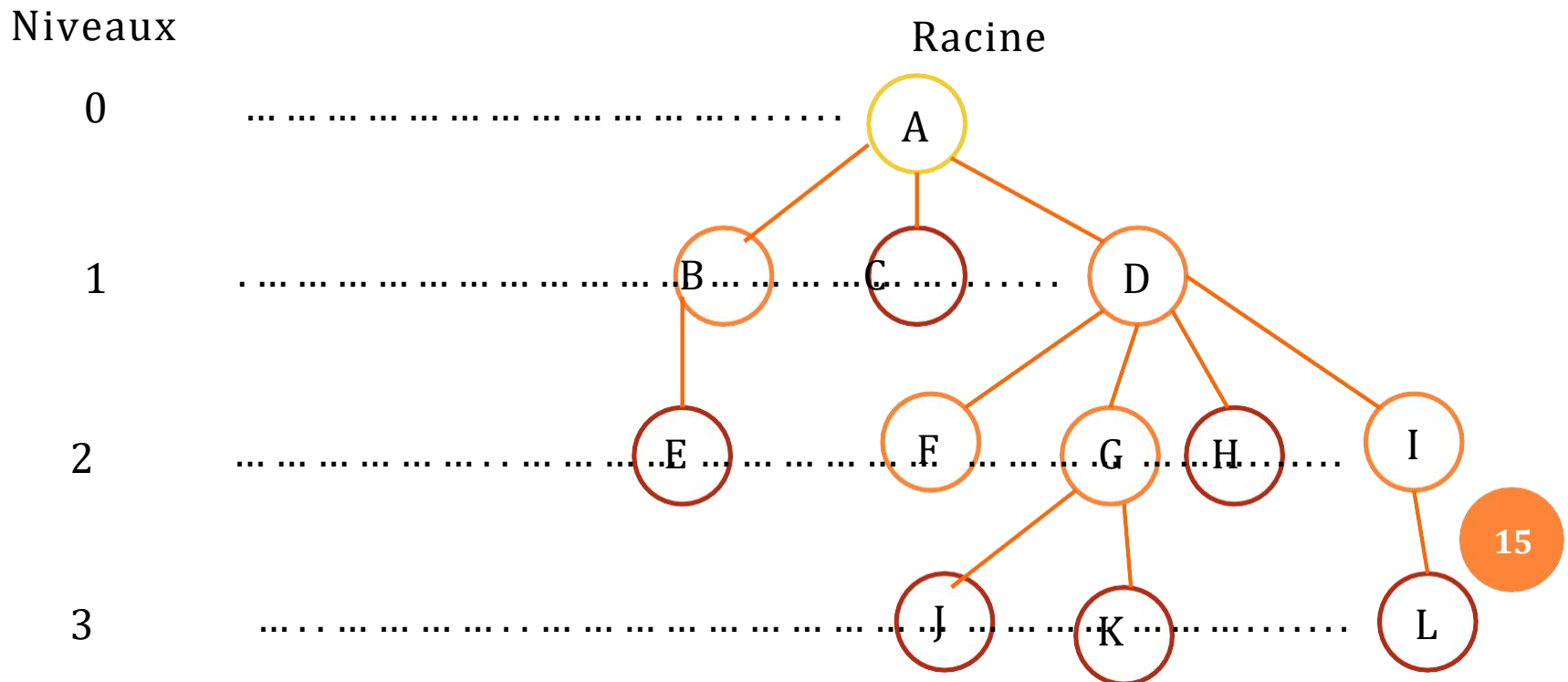
❖ **Le niveau d'un nœud:** est la distance qui le sépare de la racine:

- Le niveau de la racine = 0
- Le niveau de chaque nœud est égale au niveau de son père plus 1
- Le niveau du nœud contenant 'G' est égal à 2.



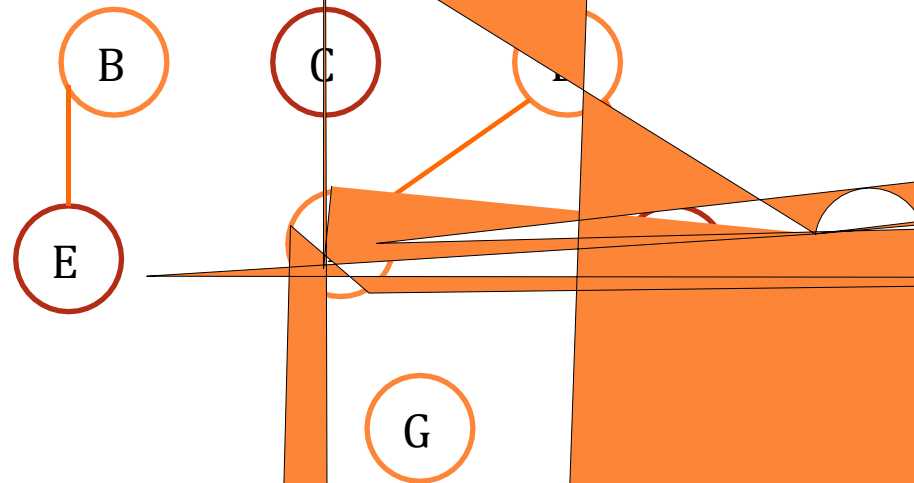
# DÉFINITIONS & TERMINOLOGIES

❖ **La profondeur d'un arbre (ou sa hauteur)** : est le plus grand niveau, c-à-d la distance entre la racine et la feuille la plus lointaine. Dans l'exemple, la profondeur de l'arbre est égal à 3



# DÉFINITIONS & TERMI

❖ **Forêt** : espace





# DÉFINITIONS & TERMINOLOGIES

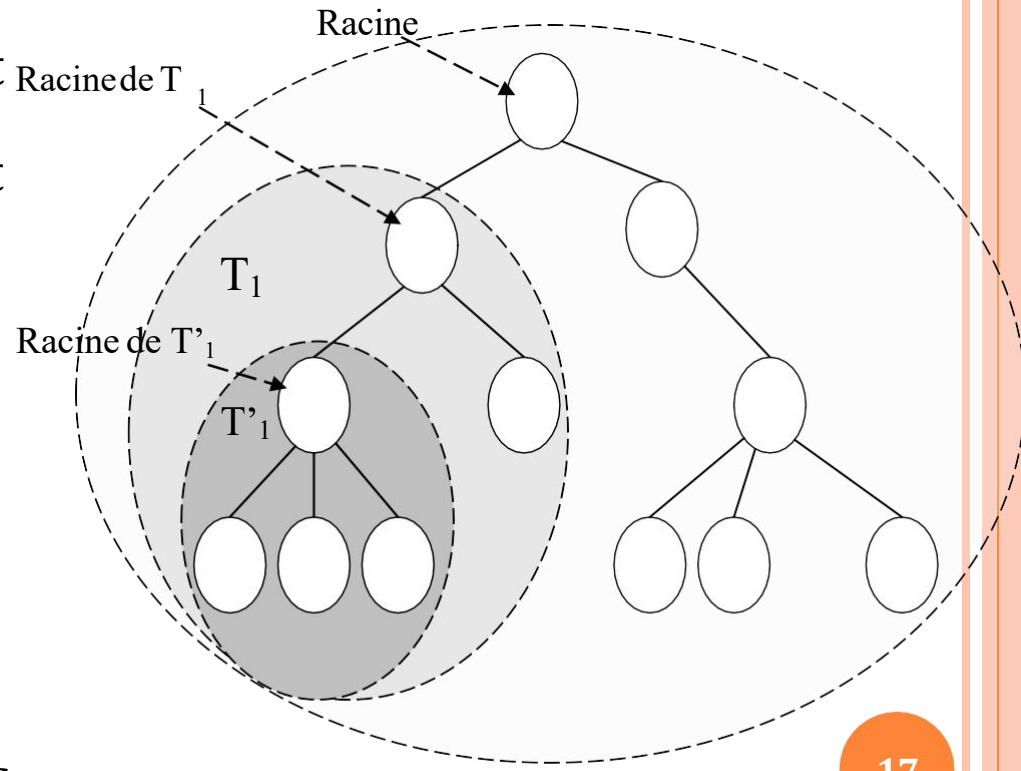
## ❖ Définition réursive

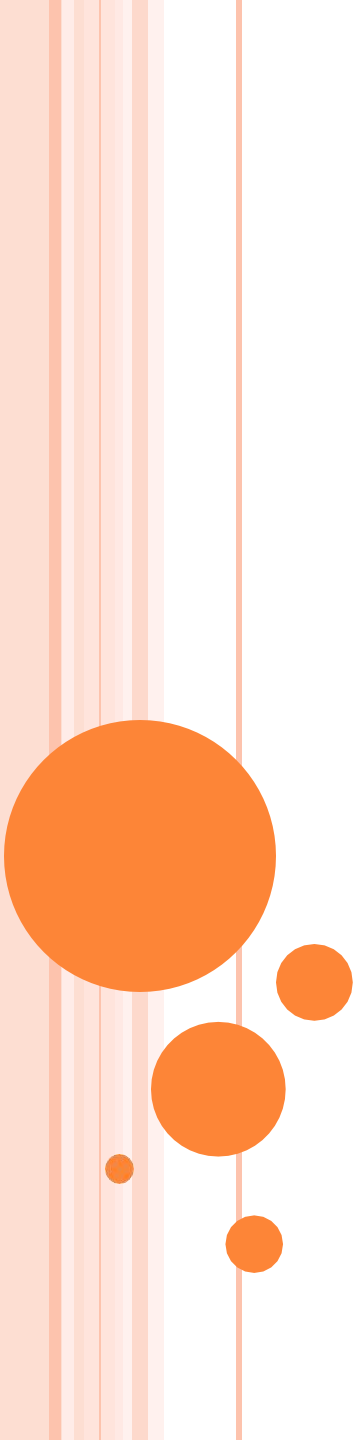
➤ Cas particulier: NIL est un arbre vide, contenant zéro nœud

➤ Cas général: SI  $n$  est un nœud et si  $T_1, T_2, \dots, T_m$  sont des arbres, ALORS on peut construire un nouvel arbre en connectant  $T_1, T_2, \dots, T_m$  comme des fils à  $n$ .

■ Chaque  $T_i$  est défini de la même manière (récursivement).

■  $T_1, T_2, \dots, T_m$  sont alors des sous-arbres de  $n$ .





# **PARTIE II:**

## **ARBRES BINAIRES**

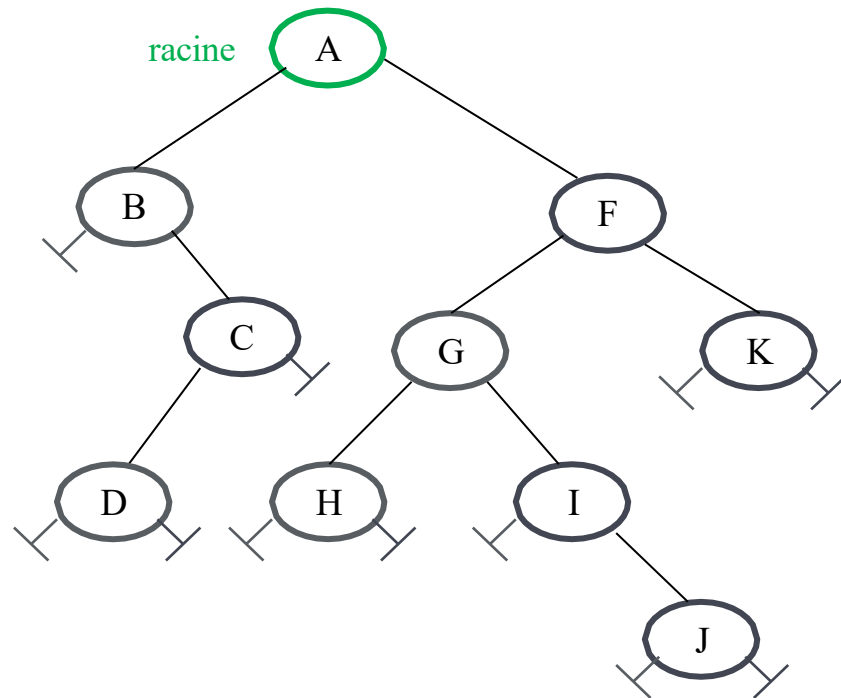
# PLAN DE LA PARTIE II

- ❑ Définition
- ❑ Modèle
- ❑ Parcours
- ❑ Représentation contigüe
- ❑ Exemple d'application

# DÉFINITION

❖ Un arbre binaire est un arbre où chaque nœud est connecté à deux sous-arbres (un sous-arbre gauche et un sous-arbre droit). Donc, un arbre binaire est un arbre de degré 2, c'est-à-dire que chaque nœuds a au plus deux fils. Ainsi, le premier fils d'un nœud n est appelé Fils-Gauche (FG) et le deuxième fils est appelé Fils-Droit (FD).

NIL  
/



# DÉFINITION

❖ Un arbre binaire est dit **strictement binaire** si chaque nœud interne (non feuille) a exactement 2 fils différents de NIL.

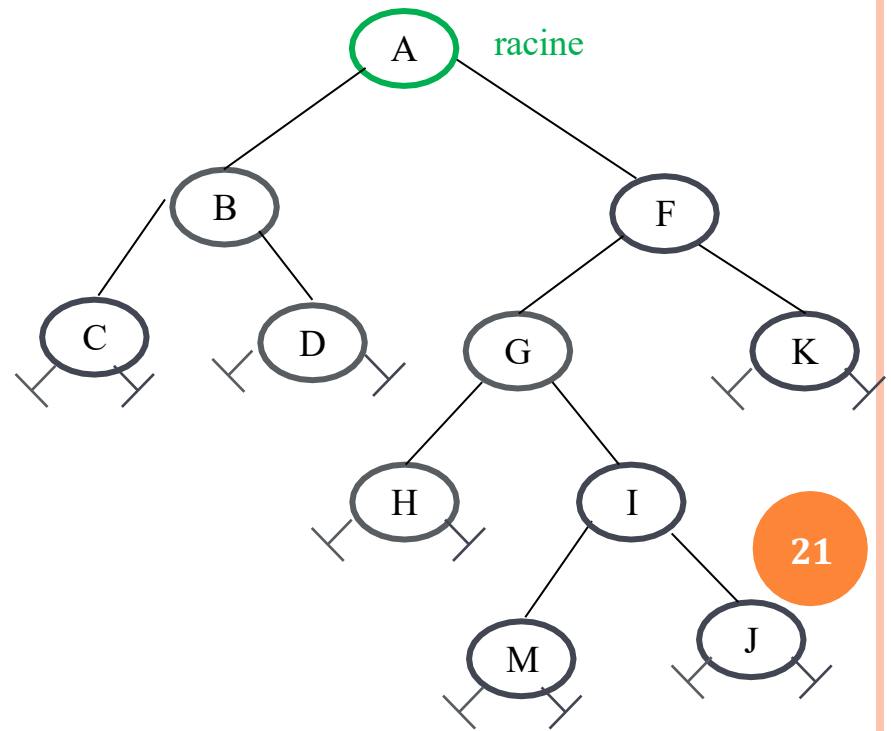
➤ Si un arbre strictement binaire a  $n$  feuilles Alors :

- le nombre total de ses nœuds =  $2n-1$ .
- le nombre de ses nœuds non feuilles (nœuds internes) =  $n-1$

➤ Le nombre de feuilles :  $n=6$   
(C, D, H, M, J, K)

➤ Le nombre total de nœuds :  
 $2n-1=11$

➤ Le nombre de nœuds  
internes:  $n-1=5$  (A, B, F, G, I)

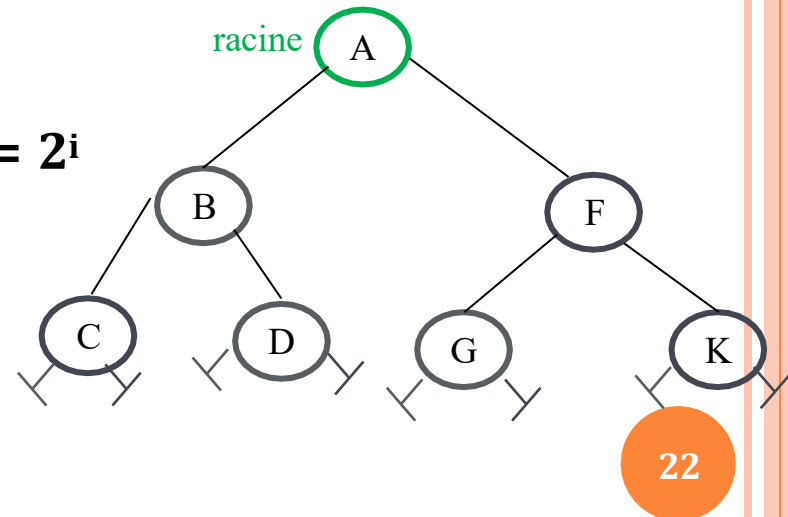


# DÉFINITION

❖ Un arbre binaire complet est un arbre strictement binaire où toutes les feuilles sont au même niveau.

❖ Dans un arbre binaire complet de profondeur « d » :

- le nombre total de nœuds  $n = 2^0 + 2^1 + 2^2 + \dots + 2^d = 2^{d+1} - 1$
- ainsi,  $d = \log_2(n+1) - 1$
- le nombre de nœuds internes =  $2^d - 1$
- le nombre de feuilles =  $2^d$
- le nombre de nœuds dans le niveau  $i = 2^i$

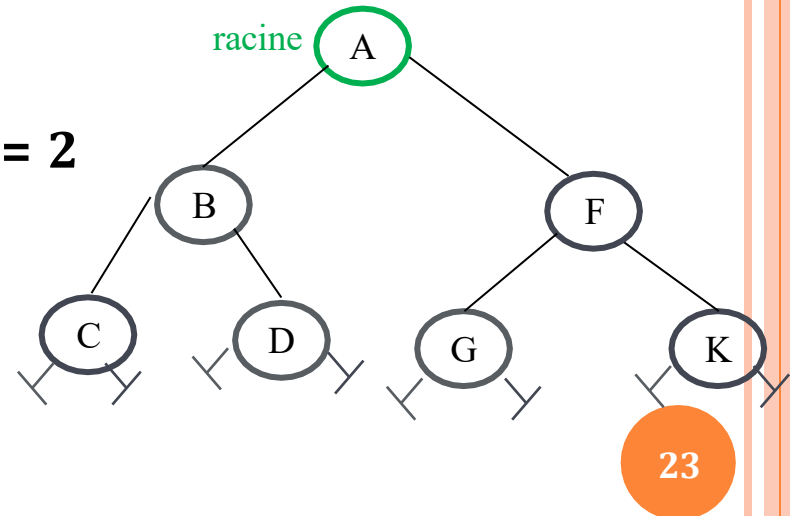


# DÉFINITION

❖ Un arbre binaire complet est un arbre strictement binaire où toutes les feuilles sont au même niveau.

❖ Dans l'exemple ci dessous:

- $d = 2$
- le nombre total de nœuds  $n = 2^3 - 1 = 7$
- le nombre de nœuds internes =  $2^2 - 1 = 3$
- le nombre de feuilles =  $2^2 = 4$
- le nombre de nœuds dans le niveau 1 = 2



# MODÈLE

- ❖ L'arbre est implémenté souvent de manière dynamique comme un ensemble de maillons (nœuds) chaînés entre eux.
- ❖ La structure d'un nœud de l'arbre est la suivante :

## Structure de Données

```
TYPE Tnoeud = STRUCTURE  
  Info : Typeqq  
  FG : * Tnoeud  
  FD : * Tnoeud  
FIN  
VAR Arbre : * Tnoeud
```



# MODÈLE

❖ On définit le modèle (machine abstraite) suivant d'un arbre binaire:

| Fonction         | Rôle                                                                                                                              |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Info(p)          | permet d'accéder à l'information du nœud p                                                                                        |
| FG(p)            | permet d'accéder à l'information de fils gauche du nœud p                                                                         |
| FD(p)            | permet d'accéder à l'information de fils droit du nœud p                                                                          |
| Aff_info(p, x)   | permet de modifier l'information du nœud p                                                                                        |
| Aff_FG(p, q)     | permet de modifier l'adresse de fils gauche du nœud p                                                                             |
| Aff_FD(p, q)     | permet de modifier l'adresse de fils droit du nœud p                                                                              |
| Créer_noeud(x)   | permet de créer un nœud avec x comme information et retourne la référence du nœud. Le nœud créé a Nil comme fils gauche et droit. |
| Liberer_noeud(p) | permet de libérer le nœud référencé par p.                                                                                        |

# EXERCICE 01

❖ Ecrire des algorithmes récursifs pour déterminer dans un arbre binaire contenant des entiers:

- a. le nombre de nœuds,
- b. le nombre de feuilles,
- c. le nombre des nœuds internes
- d. la profondeur.
- e. la somme des contenus de tous les nœuds,
- f. le minimum des valeurs contenues.
- g. le maximum des valeurs contenues
- h. l'existence d'un élément  $x$

# PARCOURS

- ❖ Le parcours d'un arbre consiste à passer par tous ses nœuds.
- ❖ Les parcours permettent d'effectuer tout un ensemble de traitement sur les arbres.
- ❖ On distingue deux types de parcours :
  - Des parcours en **profondeur** (depth-first) explorent l'arbre branche par branche. Parmi lesquels: le Préordre, l'Inordre et le Postordre.
  - Des parcours en **largeur** (breadth-first) explorent l'arbre niveau par niveau

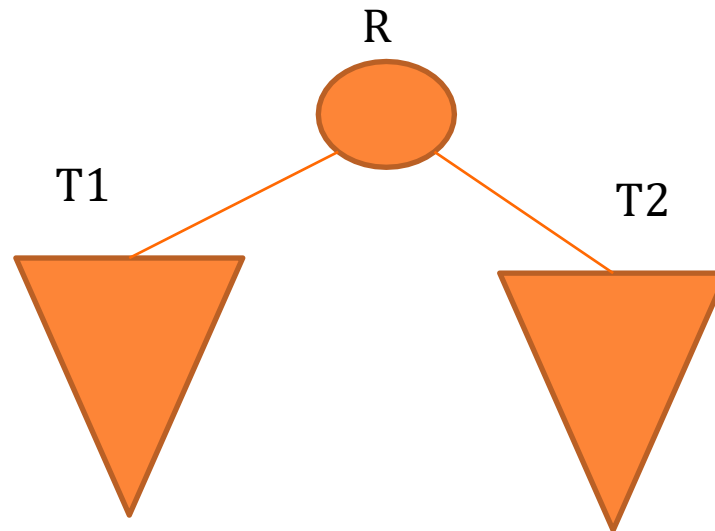
# PARCOURS EN PROFONDEUR

- ❖ Dans un parcours en profondeur, on descend le plus profondément possible dans l'arbre puis, une fois qu'une feuille a été atteinte, on remonte pour explorer les autres branches en commençant par la branche « la plus basse » parmi celles non encore parcourues.
- ❖ Le parcours en profondeur peut se faire en :
  - ❖ **Préordre (Préfixe)** : où on affiche la racine avant ses fils (Racine FG FD),
  - ❖ **Inordre (Infixe)** : où on affiche le fils gauche avant sa racine et son frère droit (FG Racine FD),
  - ❖ **Postordre (Postfixe)** : où on affiche les fils avant leur racine (FG FD Racine).

# PARCOURS EN PROFONDEUR

❖ Ces parcours (préordre, inordre et postordre) sont des parcours simples à définir et à programmer (en récursif).

- Soit  $R$  un arbre binaire (pouvant être vide :  $R = \text{NIL}$ ).
- S'il n'est pas vide (n pointe le nœud racine), alors il a la forme suivante (avec  $T1$  et  $T2$  des sous arbres définis de la même manière que  $n$ ) :

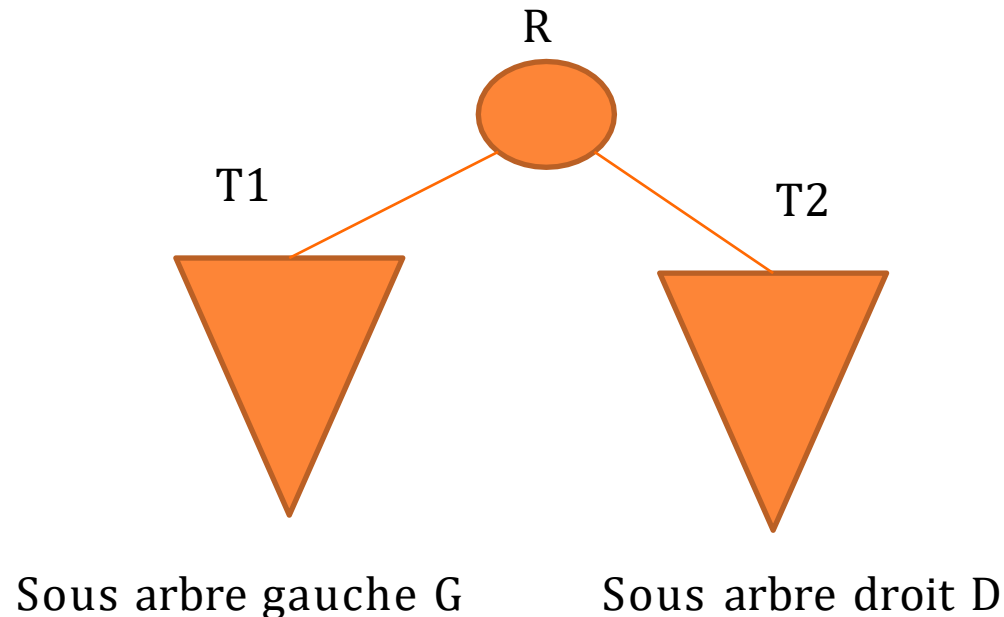


Sous arbre gauche  $G$

Sous arbre droit  $D$

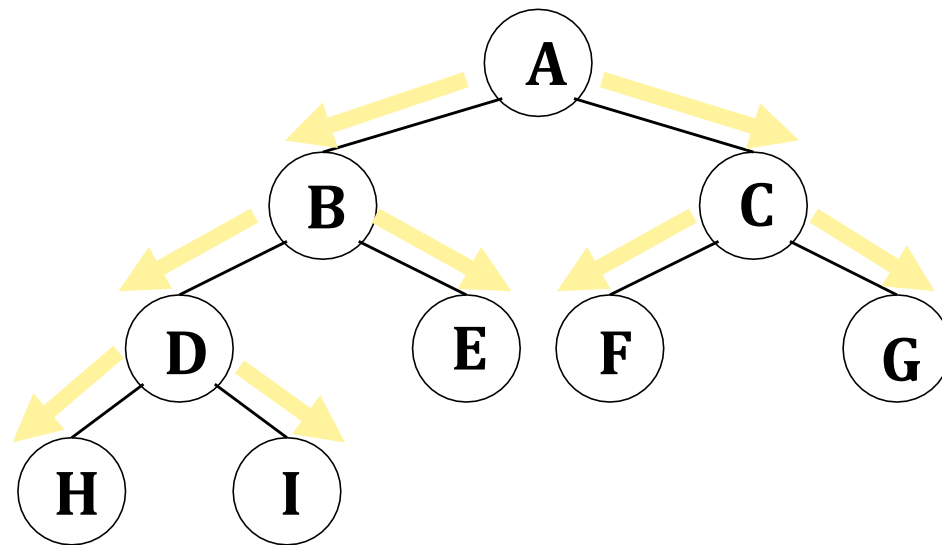
# PARCOURS PREORDRE

❖ Le parcours préordre de R (s'il n'est pas vide) consiste à visiter le nœud racine (R) ensuite parcourir récursivement en préordre les sous arbres T1 (sous arbre gauche) puis T2 (sous arbre droit) ce qui donne : **[ R , T1 , T2 ou RGD ]**



# PARCOURS PREORDRE

❖ Le parcours préordre de R (s'il n'est pas vide) consiste à visiter le nœud racine (R) ensuite parcourir récursivement en préordre les sous arbres T1 (sous arbre gauche) puis T2 (sous arbre droit) ce qui donne : **[ R , T1 , T2 ou RGD ]**



**Résultat de parcours: ABDHIEC**

**FG**

# PARCOURS PREORDRE

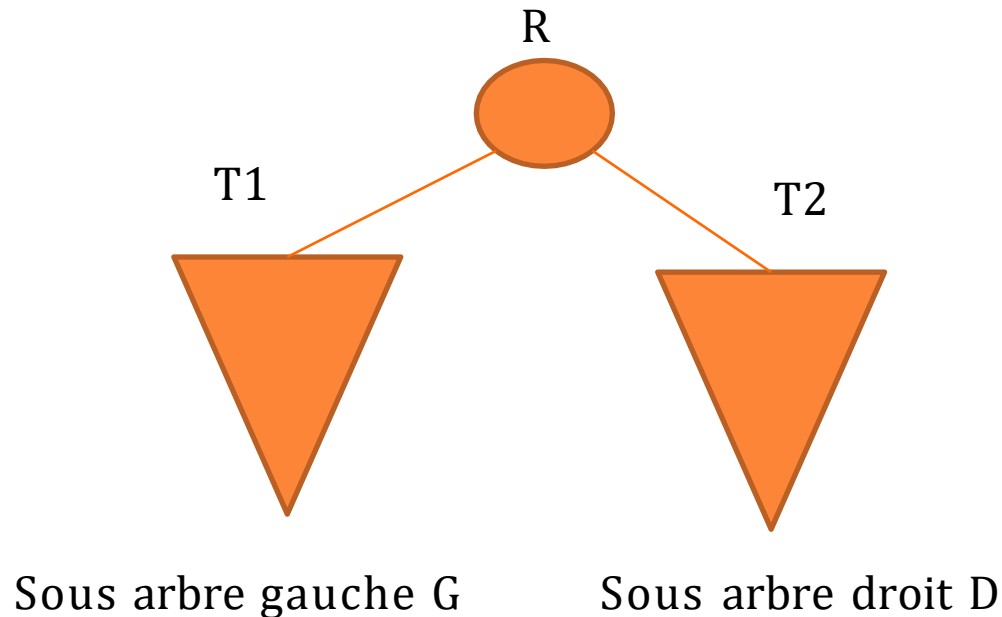
❖ La procédure (récursive) qui affiche les valeurs en parcours préordre d'un arbre de racine R est :

```
Procédure Préordre( R:* Tnoeud )  
  Début  
    SI R ≠ NIL  
      écrire( Info(R) )  
      Préordre( FG(R) )  
      Préordre( FD(R) )  
    FSI  
  fin
```



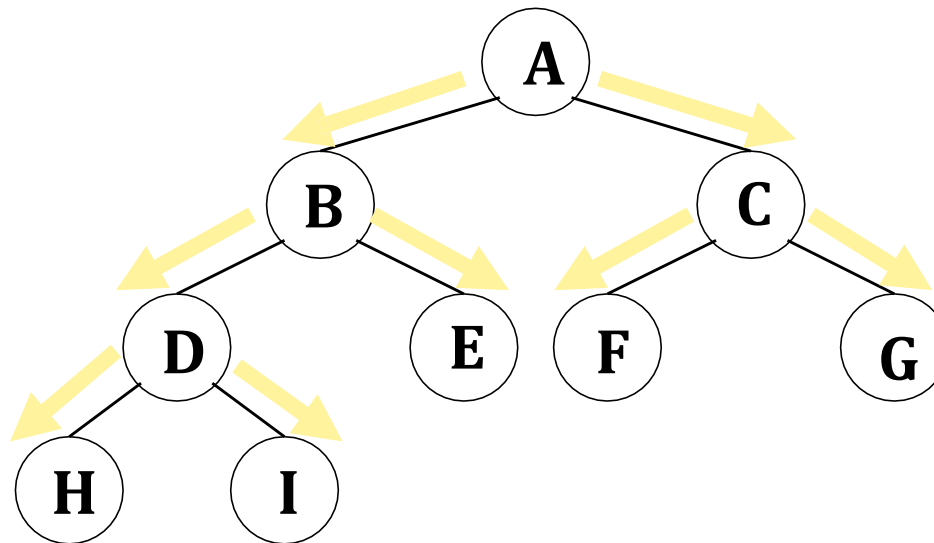
# PARCOURS INORDRE

❖ Le parcours inordre de R (s'il n'est pas vide) consiste d'abord à parcourir récursivement en inordre le sous arbre gauche T1, puis visiter le nœud racine (R) ensuite parcourir récursivement en inordre le sous arbre droit T2 ce qui donne **[ T1 , R , T2 ou GRD ]**



# PARCOURS INORDRE

❖ Le parcours inordre de R (s'il n'est pas vide) consiste d'abord à parcourir récursivement en inordre le sous arbre gauche T1, puis visiter le nœud racine (R) ensuite parcourir récursivement en inordre le sous arbre droit T2 ce qui donne **[ T1 , R , T2 ou GRD ]**



**Résultat de parcours:**

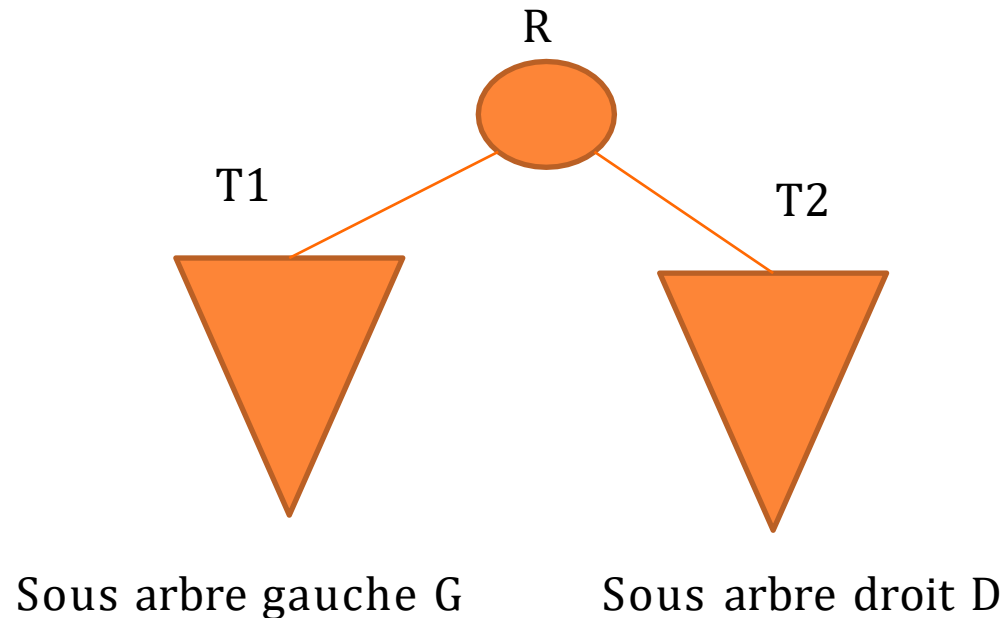
# PARCOURS INORDRE

❖ La procédure (récursive) qui affiche les valeurs en parcours inordre d'un arbre de racine R est :

```
Procédure Inordre( R:*Tnoeud )  
Début  
    SI R ≠ NIL  
        Inordre( FG(R) )  
        écrire( Info(R) )  
        Inordre( FD(R) )  
    FSI  
fin
```

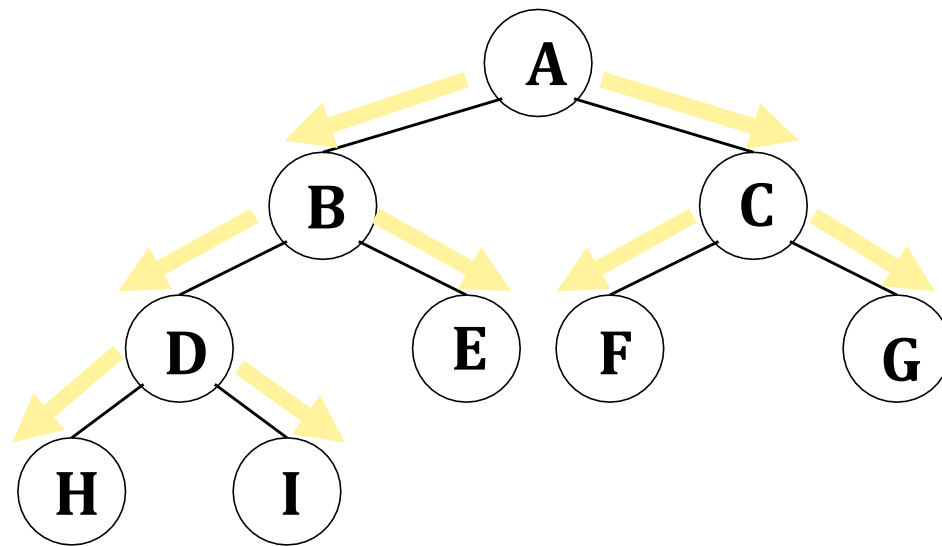
# PARCOURS POSTORDRE

❖ Le parcours postordre de R (s'il n'est pas vide) consiste d'abord à parcourir récursivement en postordre les sous arbres T1 puis T2 ensuite visiter le nœud racine (R) ce qui donne [ T1 , T2 , R ou GDR ]



# PARCOURS POSTORDRE

❖ Le parcours postordre de R (s'il n'est pas vide) consiste d'abord à parcourir récursivement en postordre les sous arbres T1 puis T2 ensuite visiter le nœud racine (R) ce qui donne [ T1 , T2 , R ou GDR]



**Résultat de parcours: HIDEBFGCA**

# PARCOURS POSTORDRE

❖ La procédure (récursive) qui affiche les valeurs en parcours postordre d'un arbre de racine R est :

```
Procédure Postordre( R: *Tnoeud)
Début
    SI R ≠ NIL
        Postordre( FG(R) )
        Postordre( FD(R) )
        écrire( Info(R) )
    FSI
fin
```

# PARCOURS EN PROFONDEUR

❖ On peut faire ces trois parcours sans utiliser la récursivité. Il faudra alors un moyen pour pouvoir remonter dans les branches de l'arbre:

➤ On pourra par exemple utiliser une structure de pile pour sauvegarder les adresses des nœuds par lesquels on est descendu et les dépiler quand on en aura besoin pour remonter.

➤ On peut aussi enrichir la structure des nœuds en incluant un pointeur vers le nœud père.

❖ Il existe d'autres types de parcours en profondeur, comme par exemple:

➤ Le préordre inverse (R T2 T1 ou RDG),

➤ L'inordre inverse (T2 R T1 ou DRG),

➤ Le postordre inverse (T2 T1 R ou DGR).

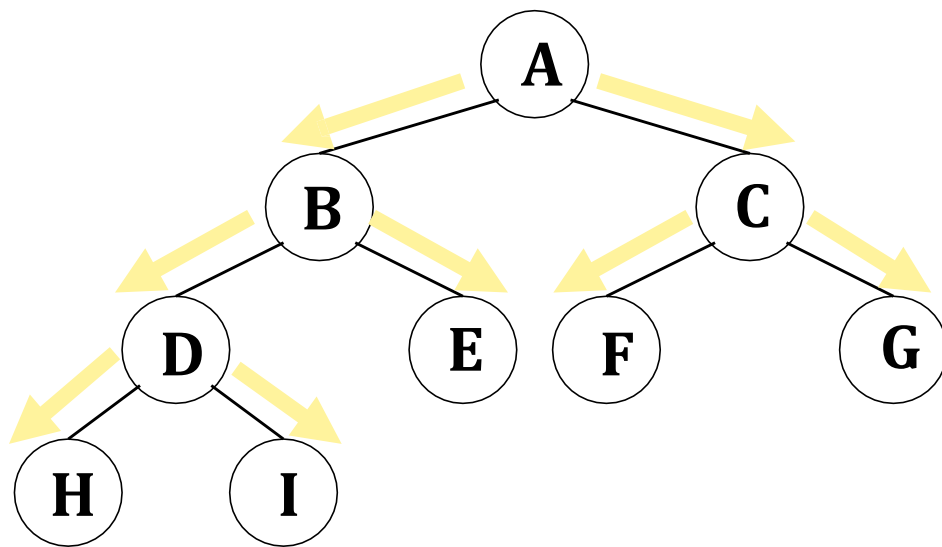
# PARCOURS EN PROFONDEUR

❖ **Exercice 02:** Trouver les algorithmes itératifs du parcours en profondeur (préordre, inordre et postordre ) dans un arbre binaire



# PARCOURS EN LARGEUR

- ❖ Dans le parcours par niveau, tous les nœuds d'un même niveau sont traités avant de descendre au niveau suivant



**Résultat de parcours:**  
ABCDEFGHI I

# PARCOURS EN LARGEUR

- ❖ Dans le parcours par niveau, tous les nœuds d'un même niveau sont traités avant de descendre au niveau suivant

```
Procédure parcours_largeur( R:* Tnoeud )  
Var F:filed'attente; P: *Tnoeud;  
Debut  
P ← R;  
Si R ≠ NIL Alors  
    Enfiler(F,P);  
    TQ (Non FileVide(F))  
        Defiler(F,P);  
        écrire(info(P));  
        Si FG(P) ≠ NIL Alors Enfiler(F,FG(P));  
        Si FD(P) ≠ NIL Alors Enfiler(F,FD(P));  
    FTQ  
Fin
```

# EXERCICE 01

❖ Ecrire des algorithmes itératifs pour déterminer dans un arbre binaire contenant des entiers:

- a. le nombre de nœuds,
- b. le nombre de feuilles,
- c. le nombre des nœuds internes
- d. la profondeur.
- e. la somme des contenus de tous les nœuds,
- f. le minimum des valeurs contenues.
- g. le maximum des valeurs contenues
- h. l'existence d'un élément  $x$

# REPRÉSENTATION CONTIGÜE

❖ On peut présenter les arbres de manière statique en utilisant les tableaux.

## ❖ Représentation Standard:

➤ Chaque élément du tableau possède quatre champs: un pour l'information, un pour le fils gauche, un pour le fils droit et un champ de type booléen pour indiquer si la case est libre ou occupée.

```
TYPE Tnoeud= STRUCTURE
```

```
Info : Typeqq
```

```
FG : entier
```

```
FD : entier
```

```
vide: booléen
```

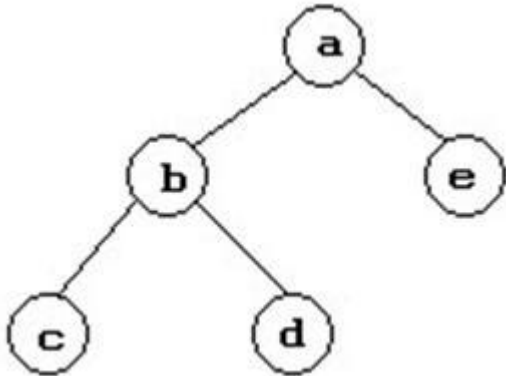
```
FIN
```

# REPRÉSENTATION CONTIGÜE

## ❖ Représentation Standard:

➤ Si la racine de l'arbre est toujours à la position 1 du tableau, l'arbre sera défini comme suit:

*VAR Arbre = TABLEAU[M] de Tnoeud*



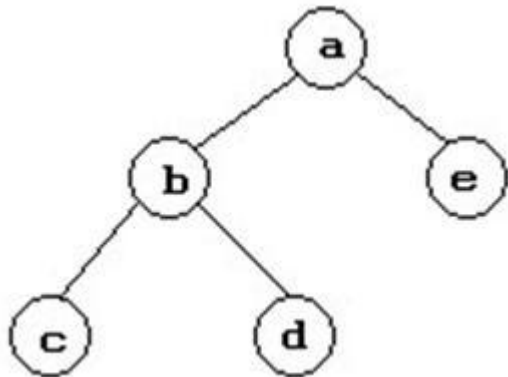
| Indice | Tnoeud |    |      |    |
|--------|--------|----|------|----|
|        | Vide   | FG | Info | FD |
| 0      | F      | 1  | a    | 4  |
| 1      | F      | 2  | b    | 3  |
| 2      | F      | -1 | c    | -1 |
| 3      | F      | -1 | d    | -1 |
| 4      | F      | -1 | e    | -1 |
| ...    | V      | ?  | ?    | ?  |
| M-1    | V      | ?  | ?    | ?  |

# REPRÉSENTATION CONTIGÜE

## ❖ Représentation Standard :

- Si on veut que la racine soit n'importe où dans le tableau, alors l'arbre sera défini (le nœud reste inchangé):

```
TYPE Tarbre = STRUCTURE  
  T : TABLEAU[M] de Tnoeud  
  Racine : ENTIER  
FIN  
VAR Arbre : Tarbre
```



Racine →

| Indice | Tnoeud |    |      |    |
|--------|--------|----|------|----|
|        | Vide   | FG | Info | FD |
| 0      | F      | -1 | c    | -1 |
| 1      | F      | 0  | b    | 3  |
| 2      | F      | 1  | a    | 4  |
| 3      | F      | -1 | d    | -1 |
| 4      | F      | -1 | e    | -1 |
| ...    | V      | ?  | ?    | ?  |
| M-1    | V      | ?  | ?    | ?  |